

This project documents the development of a basic network sniffer using Python and the Scapy library. The objective is to capture, analyze, and interpret network traffic to understand data flow and protocol structures. This practical application highlights the fundamental principles of network monitoring and vulnerability assessment, essential skills for building secure and resilient systems within the cybersecurity domain.

BASIC NETWORK SNIFFER

CodeAlpha
Internship Project |
June 2026

MOUSUMI POLLEY

Table of Contents

1. Introduction	
.....	1
2. Tools and Environment	
.....	1
3. Implementation Details	
.....	1-2
4. Demonstration	
.....	3-4
5. Challenges and Key Learnings	
.....	6-7
6. Conclusion	
.....	7
7. References	
.....	7-8

1. Introduction

The objective of this project is to develop a functional network sniffer using the Python programming language. A network sniffer is a tool used in cybersecurity to monitor and analyze traffic passing through a network interface. This project aims to provide practical experience in understanding how data packets are structured, how protocols function, and how information flows across a network.

2. Tools and Environment

- **Programming Language:** Python 3.
- **Primary Library:** scapy, used for its robust capabilities in packet sniffing, dissection, and analysis.
- **Operating System Requirements:** The script requires elevated (root or administrative) privileges to interface directly with the network hardware for raw packet capture.

3. Implementation Details

The core logic of the sniffer relies on the `sniff()` function provided by scapy, which intercepts packets in real-time. The script processes each packet using a callback function, `process_packet`,

which verifies the presence of an IP layer before extracting metadata.

PYTHON CODE:

```
from scapy.all import sniff, IP
```

```
def process_packet(packet):
```

```
    # This function runs every time a packet is  
    detected
```

```
    if packet.haslayer(IP):
```

```
        print(f'Packet: {packet[IP].src} ->  
{packet[IP].dst}')
```

```
print("Starting sniffer... Press Ctrl+C to stop.")
```

```
sniff(prn=process_packet, store=False)
```

4. Demonstration

- **Procedure:** The script was executed via the Windows Command Prompt with Administrator privileges to ensure hardware-level access. Once active, standard web browsing was performed to generate outbound and inbound network traffic.

Observation:

```
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 13.89.179.13
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 192.168.0.102 -> 13.89.179.13
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 192.168.0.102 -> 23.212.164.233
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 13.89.179.13 -> 192.168.0.102
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 13.89.179.13 -> 192.168.0.102
Packet: 192.168.0.102 -> 13.89.179.13
Packet: 192.168.0.102 -> 13.89.179.13
Packet: 192.168.0.102 -> 13.89.179.13
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 23.212.164.233
Packet: 23.212.164.233 -> 192.168.0.102
Packet: 192.168.0.102 -> 23.212.164.233
Packet: 23.212.164.233 -> 192.168.0.102
Packet: 13.89.179.13 -> 192.168.0.102
Packet: 13.89.179.13 -> 192.168.0.102
Packet: 192.168.0.102 -> 35.223.238.178
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 57.144.87.32 -> 192.168.0.102
Packet: 192.168.0.102 -> 57.144.87.32
Packet: 57.144.87.32 -> 192.168.0.102
```

Analysis: As shown in the capture, the tool successfully identified the source and destination IP addresses, demonstrating active communication between the local host (192.168.0.102) and external remote servers.

Challenges and Key Learnings

Challenges:

Permission Constraints: Packet capture requires raw socket access, necessitating that the terminal be run as an Administrator.

Path Management: Initial execution errors occurred due to the terminal defaulting to the System32 directory; this was resolved by using the cd command to navigate to the user's Desktop directory.

Key Learnings:

The project demonstrated the transparency of network metadata (IP headers), even when application-level content is secured via encryption.

It provided foundational knowledge on how network interfaces handle data frames and the importance of using industry-standard drivers like Npcap for Windows-based packet analysis.

6. Conclusion

This project successfully achieved the objective of building and testing a basic network sniffer.

Through this practical implementation, I gained hands-on experience in network traffic analysis and protocol observation, which are critical skills in the cybersecurity domain. The tools and techniques applied here form the basis for more advanced network monitoring and vulnerability assessment activities.

7. References

- Scapy Documentation. (2026). *Scapy v2.7.0 Documentation*. Retrieved from <https://scapy.readthedocs.io/>
- Nmap Project. (2026). *Npcap: Windows Packet Capture Library*. Retrieved from <https://nmap.com/>

THANK YOU !